Top 50 Java Interview Questions

Page 1 : Java Fundamentals (Q1 - Q6)

1. What are the main features of Java?

→ Simple, Object-Oriented, Platform Independent, Secure, Robust, Architecture Neutral, Portable, High Performance, Multi-threaded, Distributed.

Key Point

- Focus on Platform Independent, Secure, Robust and OOPs features in interviews.

2. Why is Java platform independent?

→ Java code is compiled into bytecode which is executed by JVM. Since JVM is available on every platform, Java becomes platform independent.

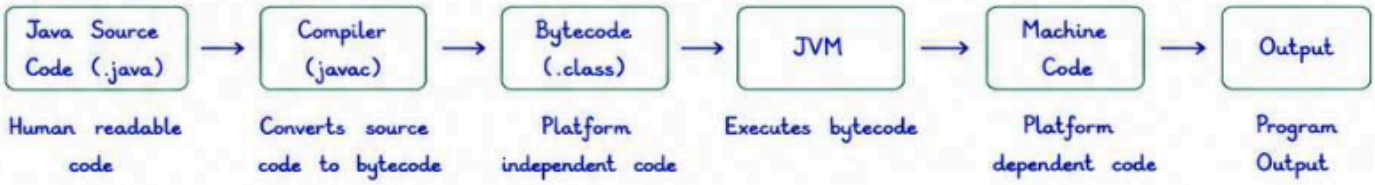
Interview Tip

- Write Once, Run Anywhere (WORA) ✓ is possible because of JVM.

3. What is the difference between JDK, JRE and JVM?

Component	Full Form	Contains	Purpose
JDK	Java Development Kit	JRE + Compiler + Tools (javac, jdb, javadoc etc.)	Used for development of Java applications.
JRE	Java Runtime Environment	JVM + Core Libraries	Used to run Java applications.
JVM	Java Virtual Machine	Class Loader, Runtime Data Areas, Execution Engine	Executes bytecode and provides runtime environment.

4. What happens when you compile and run a Java program?



5. What is the difference between primitive and non-primitive data types?

→ Primitive types hold single values and are stored in stack memory.

→ Non-primitive types hold multiple values (objects) and are stored in heap memory.

Primitive Data Types	Non-Primitive Data Types
byte, short, int, long, float, double, char, boolean	String, Arrays, Classes, Interfaces, Collections, etc.

6. What is the difference between stack and heap memory?

→ Stack memory is used for method calls, local variables and follows LIFO.

→ Heap memory is used for objects and instance variables and has no fixed order.

Stack	Heap
Stores primitive variables and reference variables. Memory is automatically managed. Faster access.	Stores objects and instance variables. Garbage Collection is used. Relatively slower access.

Top 50 Java Interview Questions

Page 2 : Java Memory & Keywords (Q7 - Q12)

7. What is the difference between stack memory and heap memory?

→ Stack memory is used for method calls, local variables and reference variables.

→ Heap memory is used for objects and instance variables.

Stack Memory	Heap Memory
Stores primitive variables and references.	Stores objects and instance variables.
Follows LIFO.	No fixed order.
Faster access.	Slower access.
Smaller size.	Larger size.

8. What is the difference between final, finally and finalize()?

final	finally	finalize()
• Keyword. • Used with variables, methods and classes. • Value cannot be changed / Overridden / Inherited.	• Block. • Used with try-catch. • Always executed whether exception occurs or not (except in abnormal termination like System.exit()).	• Method from Object class. • Called by Garbage Collector before destroying an object. • Deprecated in recent Java versions.

9. What is the static keyword and where can it be used?

→ static belongs to the class rather than any object.

→ It can be used with variables, methods, blocks and nested classes.

Where it can be used
- static variable (class variable)
- static method
- static block
- static nested class

10. What is the difference between static and instance variables?

Feature	Static Variable	Instance Variable
Belongs to	Class	Object
Memory	Allocated only once when class is loaded	Allocated each time an object is created
Access	Can be accessed using class name	Can be accessed using object reference
Default Value	Gets default value	Gets default value
Example	static int count;	int age;

11. What is pass-by-value in Java?

→ In pass-by-value, a copy of the value is passed to the method.

→ Changes made to the parameter do not affect the original value.

```
void change(int x) {
    x = 100;
}
int a = 10;
change(a);  // a is still 10
```

12. Is Java pass-by-reference or pass-by-value?

→ Java is strictly pass-by-value.

→ However, for objects, the value of reference (address) is passed by value.

→ Changes in object state will reflect, but reassigining the reference will not affect the original.

```
void modify(Student s) {
    s.name = "Aman";      // object state changed
    s = new Student();  // reassignment won't affect original
}

Student obj = new Student();
modify(obj);
System.out.println(obj.name); // Output: Aman
```

Top 50 Java Interview Questions

Page 3 : Strings & Wrapper Classes (Q13-Q18)

13. Why is String immutable in Java?

- → String is immutable to make it safe for use in multi-threaded environments.
- → It allows String Pool concept and improves performance and security.

Example

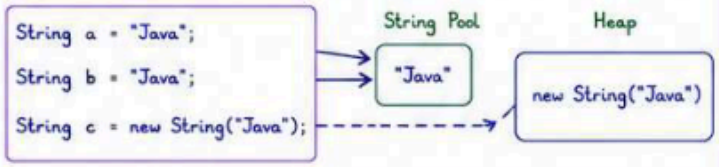
```
String s = "Hello";
s.concat(" World");
System.out.println(s);   // Output: Hello
// Original string remains unchanged
```

---

14. What is the String Pool?

- → String Pool is a special memory area in heap.
- → It stores string literals. If a literal already exists in pool, same reference is returned.

String Pool Concept



---

15. Difference between String, StringBuilder and StringBuffer?

Feature	String	StringBuilder	StringBuffer
Mutability	Immutable	Mutable	Mutable
Thread Safety	Yes (because immutable)	No	Yes (synchronized)
Performance	Slow (creates new object)	Fast	Slower than StringBuilder
Use Case	When data remains constant	Single-threaded environment	Multi-threaded environment

---

16. Difference between == and .equals() for Strings?

- → == compares reference (memory address).
- → .equals() compares actual content.

Example

```
String a = "Java";                a == b      // true (same pool reference)
String b = "Java";                a == c      // false (different object)
String c = new String("Java");    a.equals(c) // true (same content)
```

---

17. What are wrapper classes?

- → Wrapper classes are used to convert primitive types into objects.
- → They are part of java.lang package.

Primitive Type	byte	short	int	long	float	double	char	boolean
Wrapper Class	Byte	Short	Integer	Long	Float	Double	Character	Boolean

---

18. What is autoboxing and unboxing?

- → Autoboxing: Automatic conversion of primitive to wrapper class.
- → Unboxing: Automatic conversion of wrapper class to primitive.

Autoboxing	Unboxing
`int a = 10;` `Integer obj = a;  // int → Integer`	`Integer obj = 20;` `int a = obj;  // Integer → int`
JVM internally calls `Integer.valueOf(a)`	JVM internally calls `obj.intValue()`

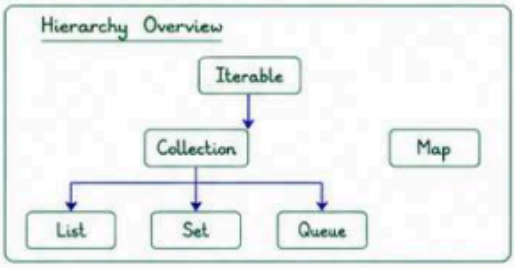
Top 50 Java Interview Questions

Page 4 : Collections Framework (Q19-Q24)

19. What is the Java Collections Framework?

→ It is a unified architecture for representing and manipulating a group of objects.

→ It provides interfaces, implementations and algorithms to work with collections.



20. Difference between List, Set and Map?

Feature	List	Set	Map
Definition	Ordered collection (index based)	Collection of unique elements	Collection of key-value pairs
Duplicates	Allows duplicates	Does not allow duplicates	Keys must be unique, values can be duplicate
Null Values	Allows multiple nulls	Allows one null	One null key allowed, multiple null values
Access	By index	No index (use iterator)	By key
Example Classes	ArrayList, LinkedList, Vector	HashSet, LinkedHashSet, TreeSet	HashMap, LinkedHashMap, TreeMap

21. Difference between ArrayList and LinkedList?

Feature	ArrayList	LinkedList
Implementation	Resizable array	Doubly linked list
Performance (Add/Remove)	Slow in middle (needs shifting)	Fast (no shifting required)
Performance (Get)	Fast (O(1))	Slow (O(n))
Memory Overhead	Less	More (extra pointers)
Use Case	Frequent access, less modification	Frequent insertion/deletion

22. Difference between HashSet, LinkedHashSet and TreeSet?

Feature	HashSet	LinkedHashSet	TreeSet
Ordering	No order	Insertion order	Sorted order (natural/custom)
Underlying Structure	Hash table	Hash table + Linked list	Red-Black Tree
Performance	Best (O(1))	Slightly slower than HashSet	Slower (O(log n))
Null Element	Allows one null	Allows one null	Does not allow null

23. Difference between HashMap, LinkedHashMap and TreeMap?

Feature	HashMap	LinkedHashMap	TreeMap
Ordering	No order	Insertion order	Sorted order (natural/custom)
Underlying Structure	Hash table	Hash table + Linked list	Red-Black Tree
Performance (Get/Put)	Best (O(1))	Slightly slower than HashMap	Slower (O(log n))
Null Key	Allows one null key	Allows one null key	Does not allow null key

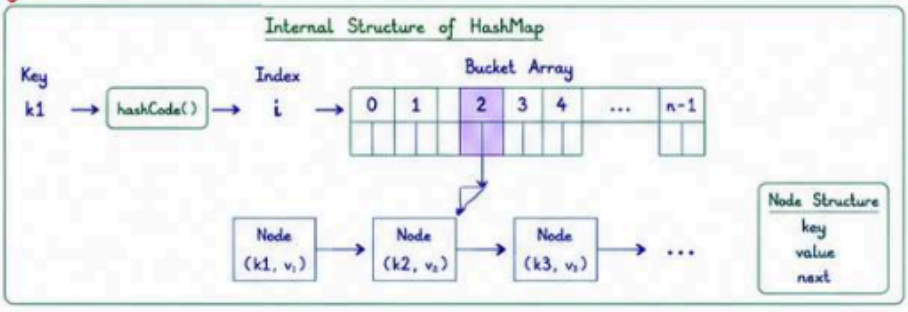
24. How does HashMap work internally?

→ HashMap stores data in key-value pairs.

→ Key's hashCode() is used to compute index.

→ Collision is handled using linked list (or tree in Java 8+ when many collisions).

→ Load factor decides when to resize (default 0.75).

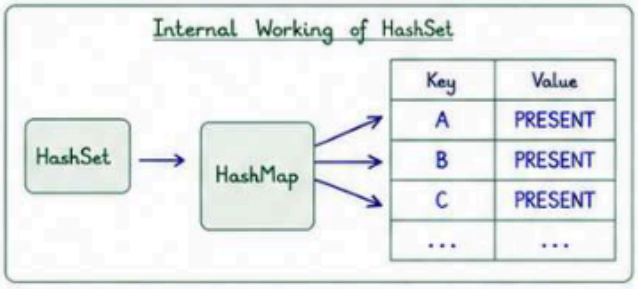


Top 50 Java Interview Questions

Page 5 : Collections Deep Dive (Q25-Q30)

25. How does HashSet work internally?

- → HashSet is backed by a HashMap internally.
- → It stores only the keys of HashMap and uses a dummy value (PRESENT).
- → Duplicate values are not allowed because of unique keys in HashMap.



26. Why are equals() and hashCode() important in collections?

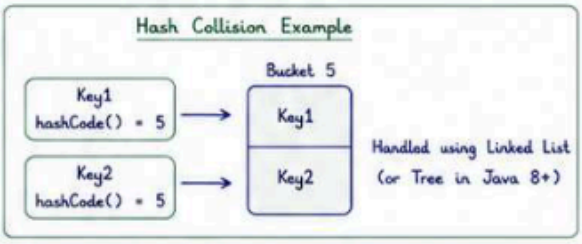
- → Collections (like HashSet, HashMap) use hashCode() to find the bucket.
- → If two objects are equal (equals()), they must have same hashCode().
- → Otherwise, duplicates may be inserted and lookup may fail.

Method	Purpose
equals()	Checks logical equality between two objects.
hashCode()	Returns integer hash code used for indexing in hash based collections.

Important Rule: If two objects are equal, they must have same hashCode().

27. What happens when two keys have the same hash code?

- → A hash collision occurs.
- → Both keys are stored in same bucket.
- → They are differentiated using equals().
- → Performance may degrade if collisions are many.



28. What is the difference between Comparable and Comparator?

Feature	Comparable	Comparator
Package	java.lang	java.util
Definition	Interface implemented by the class whose objects need natural sorting.	Interface used to define custom sorting separately.
Method	int compareTo(T o)	int compare(T o1, T o2)
Sorting	Sorts on single parameter (this object).	Sorts based on two parameters.
Use Case	Natural ordering of objects.	Custom ordering, multiple sorting logic.

29. What is an Iterator?

- → Iterator is an interface in java.util package.
- → It is used to iterate (traverse) over a collection without exposing its underlying structure.
- → It provides methods: hasNext(), next(), remove().

```
List<String> list = new ArrayList<>();
list.add("A"); list.add("B"); list.add("C");
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    String s = it.next();  // returns next element
    System.out.println(s);
}
```

30. Difference between Iterator and ListIterator?

Feature	Iterator	ListIterator
Interface	java.util.Iterator	java.util.ListIterator
Direction	One direction (forward only)	Both directions (forward & backward)
Methods	hasNext(), next(), remove()	hasNext(), next(), hasPrevious(), previous(), add(), set(), remove()
Use Case	General collection traversal	List traversal with more control

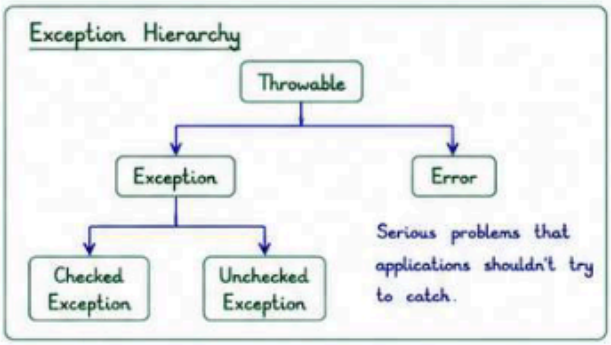
Top 50 Java Interview Questions

Page 6 : Exception Handling (Q31-Q35)

31. What is exception handling in Java?

→ Exception handling is a mechanism to handle runtime errors so that normal flow of the application can be maintained.

→ It improves the robustness of the program.



32. Difference between checked and unchecked exceptions?

Feature	Checked Exceptions	Unchecked Exceptions
Definition	Checked at compile-time.	Checked at runtime.
Superclass	Extends Exception class.	Extends RuntimeException class.
Handling	Must be handled using try-catch or throws.	Not mandatory to handle.
Examples	IOException, SQLException, ClassNotFoundException	NullPointerException, ArithmeticException, ArrayIndexOutOfBoundsException
Occurs	Due to external conditions (IO, DB, etc.)	Due to programming mistakes (logic errors)

33. Difference between throw and throws?

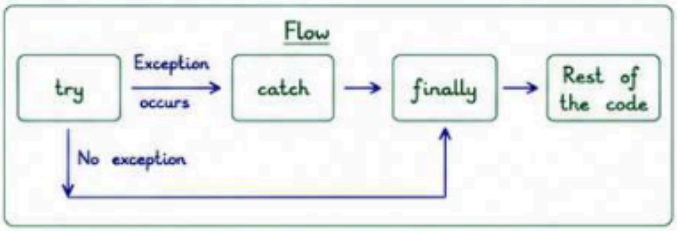
Feature	throw	throws
Purpose	Used to explicitly throw an exception.	Used to declare exceptions.
Location	Inside method/block.	In method signature.
Count	Can throw single exception at a time.	Can declare multiple exceptions.
Example	throw new ArithmeticException("Invalid number");	void readFile() throws IOException, FileNotFoundException { ... }

34. How does try-catch-finally work?

→ Code that may throw exception is written inside try block.

→ If exception occurs, catch block handles it.

→ finally block always executes whether exception occurs or not.



35. Can finally block be skipped? When?

→ Normally, finally block always executes.

→ It can be skipped only in rare cases:

- System.exit(0) is called.
- JVM crashes (out of memory, etc.).
- Power failure or OS crash.

Example

```
try {
    int a = 10 / 0;    // ArithmeticException
} catch (ArithmeticException e) {
    System.out.println("Exception: " + e);
} finally {
    System.out.println("Finally block executed");
}
```

Top 50 Java Interview Questions

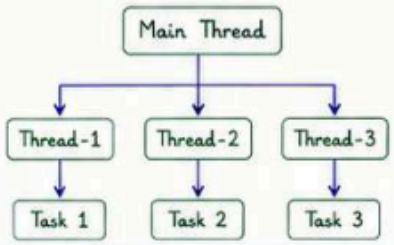
Page 7 : Multithreading & Concurrency (Q36 - Q41)

36. What is multithreading in Java?

→ Multithreading is a feature that allows concurrent execution of two or more parts of a program for maximum utilization of CPU.

→ It improves performance and responsiveness of applications.

Multithreading Concept



- All threads share same process memory.
- Threads have their own stack and program counter.
- Threads run concurrently.

37. Difference between process and thread?

Feature	Process	Thread
Definition	A program in execution.	A light-weight sub process.
Memory	Has its own independent memory.	Shares memory of the process.
Creation Time	Time consuming.	Less time consuming.
Context Switching	Costly.	Less costly.
Communication	IPC (Inter Process Communication).	Easy (Shared memory).
Examples	Chrome, Eclipse, MySQL.	Threads in a Web Server.

38. What are the ways to create a thread in Java?

① By extending Thread class.

② By implementing Runnable interface.

③ By implementing Callable interface (returns a value using Future).

④ Using ExecutorService (Thread Pool).

① Extending Thread class

```
class MyThread extends Thread {
  public void run() {
    System.out.println("Thread is running");
  }
}

MyThread t = new MyThread();
t.start();
```

② Implementing Runnable

```
class MyRunnable implements Runnable {
  public void run() {
    System.out.println("Thread is running");
  }
}

Thread t = new Thread(new MyRunnable());
t.start();
```

③ Implementing Callable

```
Callable<String> task = () -> {
  return "Completed";
};
FutureTask<String> ft = new FutureTask<>(task);
Thread t = new Thread(ft);
t.start();
System.out.println(ft.get());
```

39. Difference between start() and run()?

start()	run()
• start() creates a new thread and calls run() on that thread.	• run() is a normal method.
• It starts a new call stack.	• It does not create a new thread.
• Can be called only once on a thread.	• Can be called multiple times.
• Leads to true multithreading.	• Does not lead to multithreading.

Example

```
Thread t = new MyThread();
t.start();   // New thread
t.run();     // Just a method call
```

40. What is synchronization?

→ Synchronization is a mechanism to control access of multiple threads to shared resources.

→ It ensures data consistency and avoids thread interference.

Synchronization Example

```
class Counter {
  private int count = 0;
  public synchronized void increment() {
    count++;  // Only one thread can execute this at a time
  }
}
```

The synchronized keyword ensures that only one thread can access the method/block at a time.

41. Difference between synchronized, volatile and Atomic variables?

Feature	synchronized	volatile	Atomic (java.util.concurrent.atomic)
Purpose	Ensures mutual exclusion (one thread at a time).	Ensures visibility of changes across threads.	Ensures atomicity of operations without using locks.
Scope	Methods or blocks	Variables	Variables (atomic objects)
Guarantees	Atomicity + Visibility	Visibility only	Atomicity + Visibility
Performance	Slower (locks involved)	Faster than synchronized	Better than synchronized for simple ops
Use Case	Complex operations on shared data	When value is updated by one thread and read by others.	For counters, increment, decrement, compare-and-set operations.

@aman_views

Top 50 Java Interview Questions

Page 8 : Java 8+ & Functional Programming (Q42 – Q46)

42. What is a functional interface?

- → An interface having exactly one abstract method is called functional interface.
- → It can have any number of default, static or Object class methods.
- → Functional interfaces are used in lambda expressions.

Example

```
@FunctionalInterface
interface MyFunctionalInterface {
   void display(String msg); // only one abstract method

   default void show() {
      System.out.println("Default method");
   }
}
```

Common Functional Interfaces

Interface	Abstract Method
Runnable	void run()
Callable<V>	V call()
Comparator<T>	int compare(T o1, T o2)
Predicate<T>	boolean test(T t)
Function<T,R>	R apply(T t)
Consumer<T>	void accept(T t)
Supplier<T>	T get()

---

43. What is a lambda expression?

- → Lambda expression is a way to represent an anonymous function.
- → It provides a clear and concise way to implement functional interfaces.
- → Introduced in Java 8.

Syntax

```
(parameters) → expression
            or
(parameters) → { statements; }
```

Example

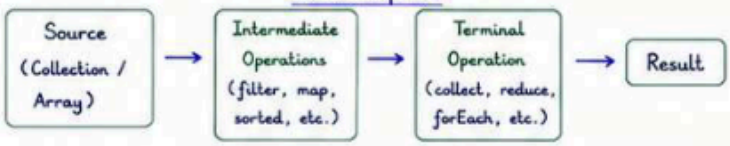
```
MyFunctionalInterface obj = (msg) → System.out.println(msg);
// or
MyFunctionalInterface obj2 = (msg) → {
    System.out.println("Hello " + msg);
};
```

---

44. What is Stream API?

- → Stream API is used to process collections of objects in a declarative way.
- → It supports functional-style operations like filter, map, reduce, etc.
- → It does not modify the original data.

Stream Pipeline



Streams are lazy – intermediate operations are executed only when a terminal operation is invoked.

---

45. Difference between map() and flatMap()?

Feature	map()	flatMap()
definition	Transforms each element and returns a new stream.	Transforms each element into a stream and flattens them.
Return Type	Stream<T>	Stream<R>
Use Case	When each input maps to single output.	When each input maps to multiple outputs (nested structure).
Example	List<String> names = list.stream() .map(String::toUpperCase) .collect(Collectors.toList());	List<String> words = list.stream() .flatMap(s → Arrays.stream(s.split(" "))) .collect(Collectors.toList());

Example

```
List<String> list = Arrays.asList("a b", "c d");
// map()
list.stream()
   .map(s → s.split(" "))
   .forEach(arr → System.out.println(Arrays.toString(arr));

// flatMap()
list.stream()
   .flatMap(s → Arrays.stream(s.split(" ")))
   .forEach(System.out::println);
```

Output: a b c d

---

46. Difference between filter(), map() and reduce()?

Operation	Purpose	Example	Return Type
filter()	Selects elements that match a condition.	list.stream() .filter(n → n > 10)	Stream<T>
map()	Transforms each element into another form.	list.stream() .map(n → n * 2)	Stream<R>
reduce()	Combines all elements to produce a single result.	list.stream() .reduce(0, (a, b) → a + b)	Optional<T> or T

Example

```
List<Integer> nums = Arrays.asList(5, 10, 15, 20);
// filter
List<Integer> filtered = nums.stream()
   .filter(n → n > 10)
   .collect(Collectors.toList()); // [15, 20]
// map
List<Integer> doubled = nums.stream()
   .map(n → n * 2)
   .collect(Collectors.toList()); // [10, 20, 30, 40]
// reduce
int sum = nums.stream()
   .reduce(0, Integer::sum);   // 50
```

Top 50 Java Interview Questions

Page 9 : Java 8+ & Functional Programming (Q47 - Q50)

47. What is Optional in Java?

- → Optional is a container object which may or may not contain a non-null value.
- → It helps to avoid NullPointerException.
- → Introduced in Java 8 in java.util package.

Example

```
Optional<String> opt = Optional.of("Aman");
if(opt.isPresent()) {
   System.out.println(opt.get());
} else {
   System.out.println("No value present");
}
```

Common Methods

Method	Description
of(value)	Creates Optional with non-null value
empty()	Creates empty Optional
isPresent()	Returns true if value is present
get()	Returns the value (if present)
orElse(value)	Returns value if present, otherwise returns default value

48. What are method references?

- → Method references are a shorthand notation of lambda expressions.
- → They make the code more readable and concise.
- → Introduced in Java 8.

Types of Method References

Type	Syntax	Example	Equivalent Lambda Expression
1. Reference to static method	Class::staticMethod	Math::max	(a, b) → Math.max(a, b)
2. Reference to instance method of a particular object	object::instanceMethod	str::toUpperCase	() → str.toUpperCase()
3. Reference to instance method of an arbitrary object of a particular type	Class::instanceMethod	String::length	s → s.length()
4. Reference to constructor	Class::new	ArrayList::new	() → new ArrayList<>()

49. What is the difference between default and static methods in interface?

Feature	Default Method	Static Method
Introduction	Introduced in Java 8	Introduced in Java 8
Belongs to	Instance of implementing class	Interface itself
Access	Through object reference	Through interface name
Override	Can be overridden	Cannot be overridden
Use	To add new functionality without breaking existing implementations	Utility/helper methods related to interface
Keyword	default	static
Example	`interface I { default void show() { System.out.println("Default method"); } }`	`interface I { static void print() { System.out.println("Static method"); } }`

Example Usage

```
I i = new MyClass();
i.show();       // default method
I.print();      // static method

class MyClass implements I {
  // can override show() if needed
}
```

50. What are best practices for using Java 8 features?

- → Use lambda expressions for small, readable blocks of code.
- → Prefer method references for better readability.
- → Use Stream API for collection processing (filter, map, reduce).
- → Use Optional to handle null values gracefully.
- → Avoid side-effects in stream operations.
- → Write clear and self-explanatory code.

Good vs Bad Example (Streams)

Good (Readable & Functional)

```
List<String> names = list.stream()
    .filter(s → s.length() > 3)
    .map(String::toUpperCase)
    .sorted()
    .collect(Collectors.toList());
```

→ Uses method reference, readable and clean.

Bad (Hard to Read)

```
List<String> names = new ArrayList<>();
for(String s : list) {
   if(s.length() > 3) {
      names.add(s.toUpperCase());
   }
}
Collections.sort(names);
```

→ More lines of code, less readable, more error prone.